

C语言指针与链表专项训练 - Day 4

参考：《C程序设计》（谭浩强）及高校C语言课程中的常考题型。

- 题型：单项选择题，共60题
- 风格：以完整或较完整的标准C程序阅读、运行结果分析和错误定位为主
- 重点：++ 与 * 的组合、指针表达式、作用域与静态变量、动态内存、字符串和链表
- 约定：除题目特别说明外，动态内存申请均成功
- 本卷不涉及时间复杂度

答题说明

每题只有一个正确答案。建议不运行程序，独立分析后作答；不确定的答案后加？，例如 8.c?，批改时会重点讲解。输出题的选项忽略行末多余空格。试卷不附答案。

一、指针、作用域与程序运行结果

1. 下列程序的输出是：

```
#include <stdio.h>

int a = 2;

int f(int n)
{
    static int a = 3;
    int t = 0;

    if (n % 2) {
        static int a = 4;
        t += a++;
    } else {
        static int a = 5;
        t += a++;
    }
    return t + a++;
}

int main(void)
{
    int s = a;
    int i;

    for (i = 0; i < 3; i++)
        s += f(i);
    printf("%d", s);
    return 0;
}
```

- A. 25
- B. 27
- C. 28
- D. 29

2. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {5, 8, 13};
    int *p = a;
    int x = *p++;
    int y = (*p)++;
    int z = *p;

    printf("%d %d %d %d", x, y, z, p == a + 1);
    return 0;
}
```

- A. 5 8 9 1
- B. 5 9 9 1
- C. 5 8 8 0
- D. 8 8 9 1

3. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {2, 4, 6};
    int *p = a;
    int x = ++*p;
    int y = *++p;

    printf("%d %d %d", x, y, a[0] + a[1]);
    return 0;
}
```

- A. 2 4 6
- B. 3 4 7
- C. 3 5 8
- D. 2 6 7

4. 下列程序的输出是：

```
#include <stdio.h>

void step(int **pp)
{
    (*pp)++;
    **pp += 10;
}

int main(void)
{
    int a[] = {1, 2, 3};
    int *p = a;

    step(&p);
    printf("%d %d %td", a[0], a[1], p - a);
    return 0;
}
```

- A. 11 2 0
- B. 1 2 1
- C. 1 12 1
- D. 11 12 0

5. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int (*p)[3] = a;

    p++;
    (*p)[1] += 10;
    printf("%d %d", p[0][1], (*(p - 1) + 2));
    return 0;
}
```

- A. 5 2
- B. 15 2
- C. 5 3
- D. 15 3

6. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    const char *words[] = {"red", "green", "blue"};
    const char **p = words + 2;

    printf("%c %s", p[-1][2], *p);
    return 0;
}
```

- A. r green
- B. e blue
- C. n blue
- D. e green

7. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    char s[] = "pointer";
    char *p = s;

    while (*p != '\0' && *p != 'n')
        p++;
    printf("%c %td", *p, p - s);
    return 0;
}
```

- A. n 3
- B. n 4
- C. i 2
- D. t 4

8. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = calloc(5, sizeof *p);
    int i, sum = 0;

    p[1] = 3;
    p[3] = 7;
    for (i = 0; i < 5; i++)
        sum += p[i];

    printf("%d %d", sum, p[4]);
    free(p);
    return 0;
}
```

- A. 10 不确定

- B. 7 0
- C. 10 0
- D. 10 5

9. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    const char *src = "link";
    char *p = malloc(strlen(src) + 1);

    strcpy(p, src);
    p[0] = 'L';
    printf("%s %zu", p, strlen(p));

    free(p);
    return 0;
}
```

- A. link 5
- B. Link 4
- C. Link 5
- D. link 4

10. 输入为一行 A B ，并在该行换行后立即结束输入。下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int ch;
    int count = 0;

    while ((ch = getchar()) != EOF) {
        if (ch != ' ')
            count++;
    }
    printf("%d", count);
    return 0;
}
```

- A. 1
- B. 2
- C. 4
- D. 3

11. 输入为：

```
12
Z
```

下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int n;
    int c1, c2;

    scanf("%d", &n);
    c1 = getchar();
    c2 = getchar();

    printf("%d %d", c1 == '\n', c2 == 'Z');
    return 0;
}
```

- A. 0 0
- B. 0 1
- C. 1 1
- D. 1 0

12. 下列程序的输出是：

```
#include <stdio.h>

int add(int x, int y)
{
    return x + y;
}

int mul(int x, int y)
{
    return x * y;
}

int apply(int (*fp)(int, int), int *x)
{
    *x = fp(*x, 2);
    return *x;
}

int main(void)
{
    int x = 3;
    int a = apply(add, &x);
    int b = apply(mul, &x);

    printf("%d %d %d", a, b, x);
    return 0;
}
```

- A. 5 10 10
- B. 5 6 6
- C. 3 10 10
- D. 5 10 3

13. 下列程序的输出是：

```
#include <stdio.h>

int inc(int x)
{
    return x + 1;
}

int twice(int x)
{
    return x * 2;
}

int main(void)
{
    int (*fp[2])(int) = {inc, twice};
    int x = 3;
    int i;

    for (i = 0; i < 2; i++)
        x = fp[i](x);

    printf("%d", x);
    return 0;
}
```

- A. 7
- B. 8
- C. 9
- D. 12

14. 下列程序的输出是：

```
#include <stdio.h>

int *counter(void)
{
    static int x = 1;

    x += 2;
    return &x;
}

int main(void)
{
    int *p = counter();
    int a = *p;
    int *q = counter();

    printf("%d %d %d", a, *p, p == q);
    return 0;
}
```

- A. 3 3 0
- B. 3 5 0
- C. 3 5 1
- D. 5 5 1

15. 下列程序存在的主要问题是：

```
#include <stdio.h>

int *make_value(void)
{
    int x = 7;

    return &x;
}

int main(void)
{
    int *p = make_value();

    printf("%d", *p);
    return 0;
}
```

- A. x 必须声明为全局变量才能参与加法
- B. 指针只能由 malloc 函数返回
- C. 程序唯一的问题是没有调用 free
- D. 函数返回后局部变量 x 的生存期结束，解引用 p 的行为未定义

二、动态内存、字符串与结构体指针

16. 下列程序的主要问题是：

```
#include <stdlib.h>

int main(void)
{
    int *p = malloc(2 * sizeof *p);
    int *q = malloc(3 * sizeof *q);

    p[0] = 1;
    q[0] = 2;

    p = q;
    p[0] = 5;

    free(p);
    return 0;
}
```

- A. 第一次分配给 p 的内存地址丢失，造成内存泄漏
- B. p[0] = 5 造成数组越界
- C. free(p) 会同时释放两块内存
- D. 指针变量之间不能相互赋值

17. 下列程序存在的主要问题是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof *p);
    int *q = p;

    *p = 9;
    free(p);
    p = NULL;

    printf("%d", *q);
    return 0;
}
```

- A. 将 p 置为 NULL 会同时把 q 置为 NULL
- B. q 成为悬空指针，解引用 q 的行为未定义
- C. q 仍然有效并且一定输出9
- D. 应改为只执行 free(q) 和 free(p)

18. 下列程序的正确修改是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(2 * sizeof *p);
    int *alias = p;
    int *q;

    p[0] = 3;
    q = realloc(p, 4 * sizeof *q);
    q[0] = 7;

    printf("%d", *alias);
    free(q);
    return 0;
}
```

- A. 原程序一定输出7，不需要修改
- B. 在 realloc 后执行 alias++
- C. 将输出语句改为 printf("%d", q[0]);
- D. 在程序末尾同时执行 free(alias); free(q);

19. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int **m = malloc(2 * sizeof *m);
    int i, j;

    for (i = 0; i < 2; i++)
        m[i] = malloc(3 * sizeof **m);

    for (i = 0; i < 2; i++)
        for (j = 0; j < 3; j++)
            m[i][j] = i * 10 + j;

    printf("%d %d", m[1][2], *((m + 1) + 1));

    for (i = 0; i < 2; i++)
        free(m[i]);
    free(m);
    return 0;
}
```

- A. 2 1
- B. 11 12
- C. 12 2
- D. 12 11

20. 下列程序的输出是：

```
#include <stdio.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void step(Node **pp)
{
    if (*pp != NULL)
        *pp = (*pp)->next;
}

int main(void)
{
    Node n3 = {3, NULL};
    Node n2 = {2, &n3};
    Node n1 = {1, &n2};
    Node *p = &n1;

    step(&p);
    p->data += 5;

    printf("%d %d %d", n1.data, n2.data, p->data);
    return 0;
}
```

- A. 1 7 7
- B. 6 2 2
- C. 1 2 7
- D. 6 7 7

21. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "banana";
    char *p = strchr(s, 'n');

    *p = 'N';
    printf("%s %td", s, p - s);
    return 0;
}
```

- A. baNana 3
- B. banana 2
- C. baNana 2
- D. banaNa 4

22. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[8] = "abc";

    memmove(s + 1, s, strlen(s) + 1);
    s[0] = 'X';

    printf("%s", s);
    return 0;
}
```

- A. Xabc
- B. Xbc
- C. aabc
- D. 行为未定义

23. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a = 1, b = 2, c = 3;
    int *p[3] = {&a, &b, &c};

    *p[1] += *p[0];
    p[0] = p[2];

    printf("%d %d %d", a, b, *p[0]);
    return 0;
}
```

- A. 3 3 1
- B. 1 2 3
- C. 3 2 3
- D. 1 3 3

24. 下列程序的输出是：

```
#include <stdio.h>

void change(int *p, int *q)
{
    int t;

    (*p)++;
    t = *p;
    *q += t;
}

int main(void)
{
    int x = 2;

    change(&x, &x);
    printf("%d", x);
    return 0;
}
```

- A. 3
- B. 6
- C. 4
- D. 5

25. 下列程序的输出是：

```
#include <stdio.h>

int *pick(int n)
{
    static int odd = 1;
    static int even = 10;

    if (n % 2) {
        odd += n;
        return &odd;
    }

    even -= n;
    return &even;
}

int main(void)
{
    int i;
    int sum = 0;

    for (i = 0; i < 4; i++) {
        int *p = pick(i);
        sum += *p;
    }

    printf("%d", sum);
    return 0;
}
```

- A. 21
- B. 23
- C. 25
- D. 27

三、链表程序阅读与改错

第26—50题默认已包含 `stdio.h`、`stdlib.h`，且动态内存申请成功。第26—39题及第43—50题使用以下定义：

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

26. 下列程序的输出是：

```
int state(Node *head)  
{  
    if (head == NULL)  
        return 0;  
    if (head->next == NULL)  
        return 1;  
    return 2;  
}  
  
int main(void)  
{  
    Node a = {7, NULL};  
    Node *head = NULL;  
  
    printf("%d ", state(head));  
    head = &a;  
    printf("%d", state(head));  
    return 0;  
}
```

- A. 0 0
- B. 0 1
- C. 1 0
- D. 1 1

27. 下列程序的输出是：

```
void append(Node **head, Node **tail, int x)
{
    Node *n = malloc(sizeof *n);
    n->data = x;
    n->next = NULL;

    if (*tail == NULL)
        *head = n;
    else
        (*tail)->next = n;
    *tail = n;
}

int main(void)
{
    Node *head = NULL, *tail = NULL, *p;
    int i, sum = 0;

    for (i = 1; i <= 3; i++)
        append(&head, &tail, i * i);
    for (p = head; p != NULL; p = p->next)
        sum += p->data;
    printf("%d %d %d", head->data, tail->data, sum);
    return 0;
}
```

- A. 1 9 9
- B. 9 1 14
- C. 1 9 14
- D. 1 9 10

28. 下列程序的输出是：

```
Node *find(Node *head, int x)
{
    while (head != NULL && head->data != x)
        head = head->next;
    return head;
}

int main(void)
{
    Node c = {6, NULL};
    Node b = {4, &c};
    Node a = {2, &b};
    Node *p, *q = find(&a, 4);

    if (q != NULL)
        q->data += 5;
    for (p = &a; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 2 4 6
- B. 2 5 6
- C. 9 4 6
- D. 2 9 6

29. 下列程序的输出是：

```
Node *make_node(int x, Node *next)
{
    Node *n = malloc(sizeof *n);
    n->data = x;
    n->next = next;
    return n;
}

void erase_first(Node **head, int x)
{
    Node **link = head;

    while (*link != NULL && (*link)->data != x)
        link = &(*link)->next;
    if (*link != NULL) {
        Node *q = *link;
        *link = q->next;
        free(q);
    }
}

int main(void)
{
    Node *p;
    Node *head = make_node(1,
                          make_node(2, make_node(3, NULL)));

    erase_first(&head, 1);
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 2 3
- B. 1 2 3
- C. 1 3
- D. 空表

30. 已知 head 是带头结点链表 2→2→3→2 的头结点，数据结点均由 malloc 申请。下列代码的输出是：

```
int erase_all(Node *head, int x)
{
    Node *p = head;
    int count = 0;

    while (p->next != NULL) {
        if (p->next->data == x) {
            Node *q = p->next;
            p->next = q->next;
            free(q);
            count++;
        } else {
            p = p->next;
        }
    }
    return count;
}

int main(void)
{
    Node head = {0, NULL};
    Node *a = malloc(sizeof *a), *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c), *d = malloc(sizeof *d);
    int n;

    a->data = 2; a->next = b;
    b->data = 2; b->next = c;
    c->data = 3; c->next = d;
    d->data = 2; d->next = NULL;
    head.next = a;
    n = erase_all(&head, 2);
    printf("%d:%d", n, head.next->data);
    return 0;
}
```

- A. 2:3
- B. 3:2
- C. 3:3
- D. 4:空表

31. 下列程序的输出是：

```
void insert_ordered(Node **head, int x)
{
    Node **link = head;
    Node *n = malloc(sizeof *n);

    while (*link != NULL && (*link)->data < x)
        link = &(*link)->next;
    n->data = x;
    n->next = *link;
    *link = n;
}

int main(void)
{
    Node c = {5, NULL};
    Node b = {3, &c};
    Node a = {1, &b};
    Node *head = &a, *p;

    insert_ordered(&head, 4);
    insert_ordered(&head, 0);
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 1 3 5 4 0
- B. 0 4 1 3 5
- C. 1 3 4 5 0
- D. 0 1 3 4 5

32. 下列程序的输出是：

```
void add_after_last(Node **head, int key, int x)
{
    Node *p = *head, *last = NULL;
    Node *n;

    while (p != NULL) {
        if (p->data == key)
            last = p;
        p = p->next;
    }

    n = malloc(sizeof *n);
    n->data = x;
    if (last != NULL) {
        n->next = last->next;
        last->next = n;
    } else {
        n->next = *head;
        *head = n;
    }
}

int main(void)
{
    Node e = {5, NULL}, d = {2, &e};
    Node c = {4, &d}, b = {2, &c};
    Node *head = &b, *p;

    add_after_last(&head, 2, 9);
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 9 2 4 2 5
- B. 2 4 2 9 5
- C. 2 9 4 2 5
- D. 2 4 9 2 5

33. 下列程序的输出是：

```
Node *reverse_k(Node *head, int k)
{
    Node *tail, *p, *prev = NULL, *next;
    int i = 0;

    if (head == NULL || k <= 0)
        return head;
    tail = head;
    p = head;
    while (p != NULL && i < k) {
        next = p->next;
        p->next = prev;
        prev = p;
        p = next;
        i++;
    }
    tail->next = p;
    return prev;
}

int main(void)
{
    Node e = {5, NULL}, d = {4, &e}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b}, *p;
    Node *head = reverse_k(&a, 3);

    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 3 2 1 4 5
- B. 5 4 3 2 1
- C. 3 2 1
- D. 1 2 3 4 5

34. 下列程序最终输出两条链表，输出结果是：

```
void split(Node *head, Node **odd, Node **even)
{
    Node *p, *q;

    *odd = head;
    *even = head == NULL ? NULL : head->next;
    p = *odd;
    q = *even;

    while (q != NULL && q->next != NULL) {
        p->next = q->next;
        p = p->next;
        q->next = p->next;
        q = q->next;
    }
    if (p != NULL)
        p->next = NULL;
}

void show(Node *p)
{
    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
}
```

已知调用前 head 指向 1→2→3→4→5→NULL，执行：

```
Node *odd, *even;
split(head, &odd, &even);
show(odd);
putchar('|');
show(even);
```

- A. 123|45
- B. 24|135
- C. 135|24
- D. 15|234

35. 下列程序的输出是：

```
Node *join(Node *a, Node *b)
{
    Node **link = &a;

    while (*link != NULL)
        link = &(*link)->next;
    *link = b;
    return a;
}

int main(void)
{
    Node b2 = {8, NULL};
    Node b1 = {7, &b2};
    Node *p;
    Node *head = join(NULL, &b1);

    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 78
- B. 空表
- C. 87
- D. 程序必然崩溃

36. 下列程序的输出是：

```
int sum_circle(Node *head)
{
    Node *p;
    int sum = 0;

    for (p = head->next; p != head; p = p->next)
        sum += p->data;
    return sum;
}

int main(void)
{
    Node head = {0, NULL};
    Node a = {1, NULL}, b = {2, NULL}, c = {3, NULL};

    head.next = &a;
    a.next = &b;
    b.next = &c;
    c.next = &head;
    printf("%d", sum_circle(&head));
    return 0;
}
```

- A. 0
- B. 1
- C. 3
- D. 6

37. 下列程序的输出是：

```
Node *erase_circle(Node *tail, int x)
{
    Node *prev, *cur;

    if (tail == NULL)
        return NULL;
    prev = tail;
    cur = tail->next;
    do {
        if (cur->data == x) {
            if (cur == prev) {
                free(cur);
                return NULL;
            }
            prev->next = cur->next;
            if (cur == tail)
                tail = prev;
            free(cur);
            return tail;
        }
        prev = cur;
        cur = cur->next;
    } while (cur != tail->next);
    return tail;
}

int main(void)
{
    Node *tail = malloc(sizeof *tail);
    tail->data = 5;
    tail->next = tail;
    tail = erase_circle(tail, 5);
    printf("%d", tail == NULL);
    return 0;
}
```

- A. 0
- B. 1
- C. 5
- D. 输出不确定

38. 下列程序的输出是：

```
void push_front_circle(Node **tail, int x)
{
    Node *n = malloc(sizeof *n);
    n->data = x;

    if (*tail == NULL) {
        n->next = n;
        *tail = n;
    } else {
        n->next = (*tail)->next;
        (*tail)->next = n;
    }
}

int main(void)
{
    Node *tail = NULL, *p;
    int i;

    for (i = 1; i <= 3; i++)
        push_front_circle(&tail, i);
    p = tail->next;
    do {
        printf("%d", p->data);
        p = p->next;
    } while (p != tail->next);
    return 0;
}
```

- A. 123
- B. 231
- C. 321
- D. 312

39. 下列程序的输出是：

```
int has_cycle(Node *head)
{
    Node *slow = head, *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return 1;
    }
    return 0;
}

int main(void)
{
    Node d = {4, NULL}, c = {3, &d};
    Node b = {2, &c}, a = {1, &b};

    d.next = &b;
    printf("%d", has_cycle(&a));
    return 0;
}
```

- A. 1
- B. 0
- C. 2
- D. 无限循环且没有输出

第40—42题使用以下双向链表结点定义：

```
typedef struct DNode {
    int data;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

40. 下列程序的输出是：

```
DNode *insert_after(DNode *p, int x)
{
    DNode *n = malloc(sizeof *n);
    n->data = x;
    n->prev = p;
    n->next = p->next;

    if (p->next != NULL)
        p->next->prev = n;
    p->next = n;
    return n;
}

int main(void)
{
    DNode a = {1, NULL, NULL};
    DNode b = {3, &a, NULL};
    DNode *p;

    a.next = &b;
    insert_after(&a, 2);
    for (p = &b; p != NULL; p = p->prev)
        printf("%d", p->data);
    return 0;
}
```

- A. 123
- B. 132
- C. 231
- D. 321

41. 下列程序的输出是：

```
void erase_dnode(DNode **head, DNode **tail, DNode *q)
{
    if (q->prev != NULL)
        q->prev->next = q->next;
    else
        *head = q->next;

    if (q->next != NULL)
        q->next->prev = q->prev;
    else
        *tail = q->prev;
    free(q);
}

int main(void)
{
    DNode a = {1, NULL, NULL};
    DNode *b = malloc(sizeof *b);
    DNode *head = &a, *tail = b;

    b->data = 2;
    b->prev = &a;
    b->next = NULL;
    a.next = b;
    erase_dnode(&head, &tail, b);
    printf("%d %d %d",
           head->data, tail->data, head->next == NULL);
    return 0;
}
```

- A. 1 2 0
- B. 2 1 1
- C. 1 1 1
- D. 1 1 0

42. 已保证 $q \rightarrow \text{prev} \neq \text{NULL}$ 。为了使下列函数正确地把 n 插入到双向链表结点 q 之前，空白处应填写：

```
void insert_before(DNode *q, DNode *n)
{
    n->prev = q->prev;
    n->next = q;
    q->prev->next = n;
    /* 在此处补充一条语句 */
}
```

- A. $q \rightarrow \text{next} = n;$
- B. $q \rightarrow \text{prev} = n;$
- C. $n \rightarrow \text{prev} = q;$
- D. $n \rightarrow \text{next} = q \rightarrow \text{next};$

43. 下列程序存在的主要问题是：

```
void clear(Node *p)
{
    while (p != NULL) {
        Node *q = p->next;
        free(p);
        p = q;
    }
}

int main(void)
{
    Node *shared = malloc(sizeof *shared);
    Node *a = malloc(sizeof *a);
    Node *b = malloc(sizeof *b);

    shared->data = 9;
    shared->next = NULL;
    a->next = shared;
    b->next = shared;
    clear(a);
    clear(b);
    return 0;
}
```

- A. clear(a) 不会释放任何结点
- B. 只会造成结点 a 的内存泄漏
- C. 执行 clear(a) 后，b 会自动变成 NULL
- D. 两条链共享 shared，第二次清空会访问或再次释放已经释放的结点

44. 下列程序能够输出 1 3 ，但仍存在一个主要问题。该问题是：

```
void erase_after(Node *p)
{
    if (p != NULL && p->next != NULL)
        p->next = p->next->next;
}

int main(void)
{
    Node *a = malloc(sizeof *a);
    Node *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c);
    Node *p;

    a->data = 1; a->next = b;
    b->data = 2; b->next = c;
    c->data = 3; c->next = NULL;
    erase_after(a);
    for (p = a; p != NULL; p = p->next)
        printf("%d ", p->data);
    return 0;
}
```

- A. 结点2被断开后没有 free ，造成内存泄漏
- B. 结点1被重复释放
- C. 结点3变成循环链表
- D. a 会自动变成 NULL

45. 下列程序的输出是：

```
void clear_list(Node **head)
{
    while (*head != NULL) {
        Node *q = *head;
        *head = q->next;
        free(q);
    }
}

int main(void)
{
    Node *a = malloc(sizeof *a);
    Node *b = malloc(sizeof *b);
    Node *head = a;

    a->data = 1;
    a->next = b;
    b->data = 2;
    b->next = NULL;
    clear_list(&head);
    printf("%d", head == NULL);
    return 0;
}
```

- A. 0
- B. 2
- C. 1
- D. 输出不确定

46. 已知 head 指向动态建立的链表 1→2→2→3→NULL 。 下列程序的输出是：

```
Node *remove_all(Node *head, int x)
{
    Node *next;

    if (head == NULL)
        return NULL;
    head->next = remove_all(head->next, x);
    if (head->data == x) {
        next = head->next;
        free(head);
        return next;
    }
    return head;
}

int main(void)
{
    Node *head = malloc(sizeof *head);
    Node *b = malloc(sizeof *b), *c = malloc(sizeof *c);
    Node *d = malloc(sizeof *d), *p;

    head->data = 1; head->next = b;
    b->data = 2; b->next = c;
    c->data = 2; c->next = d;
    d->data = 3; d->next = NULL;
    head = remove_all(head, 2);
    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 1223
- B. 13
- C. 123
- D. 31

47. 下列程序的输出是：

```
Node *merge(Node *a, Node *b)
{
    Node dummy = {0, NULL};
    Node *tail = &dummy;

    while (a != NULL && b != NULL) {
        if (a->data <= b->data) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }
    tail->next = a != NULL ? a : b;
    return dummy.next;
}

int main(void)
{
    Node a3 = {7, NULL}, a2 = {4, &a3}, a1 = {1, &a2};
    Node b3 = {8, NULL}, b2 = {3, &b3}, b1 = {2, &b2};
    Node *p = merge(&a1, &b1);

    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
    return 0;
}
```

- A. 147238
- B. 123847
- C. 214378
- D. 123478

48. 下列程序的输出是：

```
Node *kth_from_end(Node *head, int k)
{
    Node *fast = head, *slow = head;
    int i;

    for (i = 0; i < k; i++) {
        if (fast == NULL)
            return NULL;
        fast = fast->next;
    }
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}

int main(void)
{
    Node e = {50, NULL}, d = {40, &e}, c = {30, &d};
    Node b = {20, &c}, a = {10, &b};
    Node *p = kth_from_end(&a, 2);

    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 40
- B. 30
- C. 20
- D. -1

49. 已保证参数 q 不是尾结点。下列程序的输出是：

```
void erase_known(Node *q)
{
    Node *next = q->next;

    q->data = next->data;
    q->next = next->next;
    free(next);
}

int main(void)
{
    Node *a = malloc(sizeof *a);
    Node *b = malloc(sizeof *b);
    Node *c = malloc(sizeof *c);
    Node *d = malloc(sizeof *d);
    Node *p;

    a->data = 1; a->next = b;
    b->data = 2; b->next = c;
    c->data = 3; c->next = d;
    d->data = 4; d->next = NULL;
    erase_known(b);
    for (p = a; p != NULL; p = p->next)
        printf("%d", p->data);
    return 0;
}
```

- A. 124
- B. 123
- C. 134
- D. 14

50. 下列程序的输出是：

```
Node *first_common(Node *a, Node *b)
{
    Node *p = a, *q = b;

    while (p != q) {
        p = p != NULL ? p->next : b;
        q = q != NULL ? q->next : a;
    }
    return p;
}

int main(void)
{
    Node d = {8, NULL}, c = {7, &d};
    Node a2 = {2, &c}, a1 = {1, &a2};
    Node b1 = {3, &c};
    Node *p = first_common(&a1, &b1);

    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 2
- B. 7
- C. 3
- D. -1

四、++ 与 * 优先级专项

本部分各表达式均在有效数组范围内执行；题目说明“各选项单独执行”时，各选项互不影响。

51. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {3, 5, 7, 9};
    int *p = a + 2;
    int x = *p--;
    int y = *p;

    printf("%d %d %td", x, y, p - a);
    return 0;
}
```

- A. 7 7 2
- B. 5 5 1
- C. 7 5 1
- D. 7 3 0

52. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {3, 5, 7, 9};
    int *p = a + 2;
    int x = *--p;
    int y = *p;

    printf("%d %d %td", x, y, p - a);
    return 0;
}
```

- A. 5 5 1
- B. 7 5 1
- C. 5 7 2
- D. 3 3 0

53. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {4, 9, 14};
    int *p = a + 1;
    int x = --*p;

    printf("%d %d %td", x, a[1], p - a);
    return 0;
}
```

- A. 9 8 1
- B. 8 9 1
- C. 8 8 0
- D. 8 8 1

54. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {4, 9, 14};
    int *p = a + 1;
    int x = (*p)--;
    int y = *p;

    printf("%d %d %td", x, y, p - a);
    return 0;
}
```

- A. 8 8 1
- B. 9 8 1
- C. 9 9 0
- D. 8 9 1

55. 已知如下定义，各选项分别在相同初始状态下单独执行。数值不等于 6 的表达式是：

```
int a[] = {2, 4, 6, 8, 10};  
int *p;
```

- A. a[2]
- B. p = a + 2, *p++
- C. p = a + 1, *(p++)
- D. p = a + 1, *++p

56. 已知：

```
int a[] = {8, 9, 10};  
int *p = a + 1;
```

以下各表达式分别在上述相同初始状态下单独执行。哪个表达式执行后会使 $p \neq a + 1$ ？

- A. (*p)++
- B. ++*p
- C. --(*p)
- D. *p++

57. 下列程序的输出是：

```
#include <stdio.h>  
  
int main(void)  
{  
    int a[] = {2, 4, 6, 8};  
    int *p = a + 1;  
    int x = *p++;  
    int y = ++*p;  
    int z = (*p)--;  
  
    printf("%d %d %d %d %d",  
           x, y, z, *p, p - a);  
    return 0;  
}
```

- A. 4 6 7 6 2
- B. 4 7 6 6 2
- C. 4 7 7 6 2
- D. 4 7 7 7 3

58. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {10, 20, 30, 40};
    int *p = a + 2;
    int x = *p--;
    int y = *--p;
    int z = (*p)++;

    printf("%d %d %d %d %td",
           x, y, z, a[0], p - a);
    return 0;
}
```

- A. 30 20 10 11 1
- B. 30 10 10 11 0
- C. 30 10 11 10 0
- D. 20 10 10 11 0

59. 下列程序的输出是：

```
#include <stdio.h>

int take(int **pp)
{
    int x = *(*pp)++;

    return x;
}

int main(void)
{
    int a[] = {3, 5, 7};
    int *p = a;
    int x = take(&p);
    int y = take(&p);

    printf("%d %d %d %td",
           x, y, *p, p - a);
    return 0;
}
```

- A. 3 3 5 0
- B. 3 5 5 1
- C. 5 7 7 2
- D. 3 5 7 2

60. 下列程序的输出是：

```
#include <stdio.h>

int calc(int **pp)
{
    int x = ++**pp;

    (*pp)++;

    {
        int y = (**pp)--;
        return x + y;
    }
}

int main(void)
{
    int a[] = {1, 3, 5};
    int *p = a;
    int result = calc(&p);

    printf("%d %d %d %d %d",
           result, a[0], a[1], *p, p - a);
    return 0;
}
```

- A. 5 2 2 2 1
- B. 6 2 3 3 1
- C. 5 1 2 2 1
- D. 4 2 2 5 2

答题卡

1:	2:	3:	4:	5:	6:	7:	8:	9:	10:
11:	12:	13:	14:	15:	16:	17:	18:	19:	20:
21:	22:	23:	24:	25:	26:	27:	28:	29:	30:
31:	32:	33:	34:	35:	36:	37:	38:	39:	40:
41:	42:	43:	44:	45:	46:	47:	48:	49:	50:
51:	52:	53:	54:	55:	56:	57:	58:	59:	60: